

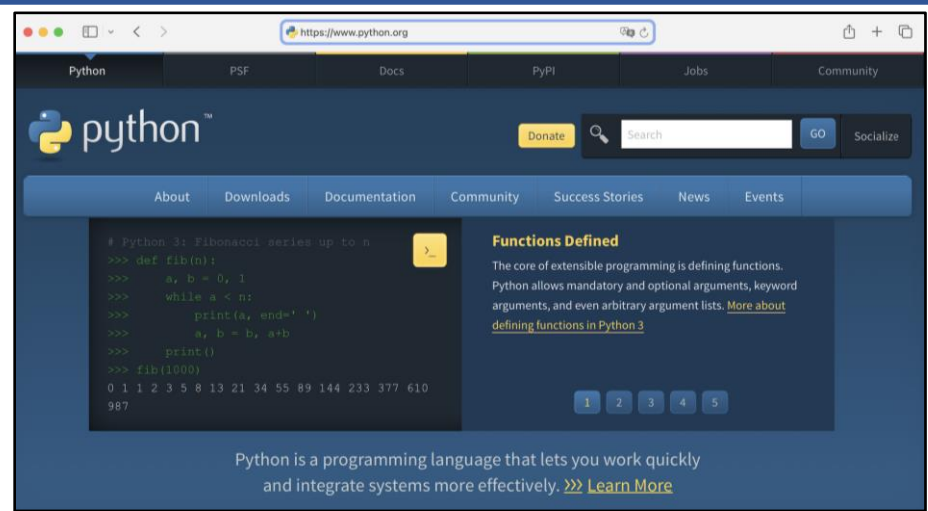
Course: Hands-On Data Analytics with Python

Eli Zamir

Lesson 2: Core Python programming (part 1)

Python main features

- General-purpose programming language
- Interpreted language (no need to compile)
→ easy to interact with the code, the data and the results
- Supports multiple programming paradigms:
 - Procedural programming
 - Object-oriented programming
 - Functional programming
- Large ecosystem of third-party libraries
- Large and active community of developers and users
- Cross-platform compatibility: Windows, macOS, Linux
- Open source and free
- High applicability across many domains
- Strong tools (Jupyter Notebook , IDEs, terminal)



Comments with Python code

```
# This is a single-line comment

speed = 70 # This is an in-line comment; km / hours

"""
This is a multi-line comment
line 2 ...
line 3 ...
"""

time = 5 # hours

'''
This is a also multi-line comment
line 2 ...
line 3 ...
'''

distance = speed * time

'''
In a multi-line comment,
the ; suppresses the display of the
comments in the Jupyter notebook output cell
'''

print(f"The distance is {distance} km.")
```

The distance is 350 km.

Data Types in Python

Python has 5 basic built-in data types:

int

Integers

```
animal_legs = 4  
  
type(animal_legs)  
  
int
```

str

Strings

```
animal_type = "Dog"  
  
type(animal_type)  
  
str
```

float

floating-point numbers

```
animal_weight = 15.23  
  
type(animal_weight)  
  
float
```

bool

Booleans

```
animal_is_dog = True  
  
type(animal_is_dog)  
  
bool
```

complex

Complex numbers

```
example_complex = 0 + 1j  
  
type(example_complex)  
  
complex
```

Variables: Containers for storing values that can change during program execution (e.g., `animal_weight`)

Data Types: The kind of value that a variable holds (e.g., `float`)

Python is a dynamically typed language

Dynamically typed language: The type of a variable is determined at runtime and can change during execution

```
a = 7
b = 7.0
c = a + b
```

```
print(f"(1) a: {a} (type: {type(a)})")
print(f"(2) b: {b} (type: {type(b)})")
print(f"(3) c: {c} (type: {type(c)})")
```

→ (1) a: 7 (type: <class 'int'>)
(2) b: 7.0 (type: <class 'float'>)
(3) c: 14.0 (type: <class 'float'>)

```
d = 4
e = 8 / 2
```

```
print(f"(4) d: {d} (type: {type(d)})")
print(f"(5) e: {e} (type: {type(e)})")
```

→ (4) d: 4 (type: <class 'int'>)
(5) e: 4.0 (type: <class 'float'>)

```
e = True
print(f"(6) e: {e} (type: {type(e)})")
```

→ (6) e: True (type: <class 'bool'>)

```
e = "Elephant"
print(f"(7) e: {e} (type: {type(e)})")
```

→ (7) e: Elephant (type: <class 'str'>)

Operators

Arithmetic operators:

+ - * / the standard arithmetic symbols

** exponentiation

% modulo division

// floor division

```
8 % 5
```

```
3
```

```
8 // 5
```

```
1
```

```
a = 8 / 2  
print(a, type(a))
```

```
4.0 <class 'float'>
```

```
b = 8 // 2  
print(b, type(b))
```

```
4 <class 'int'>
```

Comparison operators

== Equal to ?

!= Not equal to ?

> < greater than or less than ?

<= Less than or equal ?

>= Great than or equal ?

```
5 == 5
```

```
True
```

```
5 != 5
```

```
False
```

Logical operators

and Both must be True

or At least one must be True

not Reverses the Boolean state

```
(True) and (False)
```

```
False
```

```
(True) or (False)
```

```
True
```

```
not(True)
```

```
False
```

Simple Input and Output

Input: the `input()` function

```
user_name = input("Enter your name:")  
Enter your name: Mike
```

Note:

In practice, the `input()` function is more commonly used in Python scripts, although it also works in Jupyter Notebooks

Output: the `print()` function

```
user_name = input("Enter your name:")  
Enter your name: Mike  
  
print(user_name)  
Mike
```

The formatted string literal (f-string)

f-string enables to embed and evaluate Python expressions directly inside {} at runtime

`f"text{expression}"`

```
a = 7
f"a equals {a}"
'a equals 7'
```

Example usage within the `print()` function:

```
user_name = "Mike"
print(f"Hello {user_name} !")
Hello Mike !
```


The formatted string literal (f-string)

The { } can contain Python expression which are evaluated to a value:

Numerical
operations

```
length = 15
width = 3
print(f"If length = {length} cm and width = {width} cm, then area = {length * width} cm^2")
```

If length = 15 cm and width = 3 cm, then area = 45 cm^2

String
operations

```
a = "AbCdEfgH"
print(f"characters = {len(a)}, {a.upper()}, {a.lower()}")
```

characters = 8, ABCDEFGH, abcdefgh

Logical
operations

```
a = 2; b = 2; c = 3
print(f"The statment a = b is {a==b}")
print(f"The statment a = c is {a==c}")
```

The statment a = b is True
The statment a = c is False

The formatted string literal (f-string)

Escape sequences

`\n` new line

`\t` tab

```
print(f"Length\tWidth\tArea")
print(f"{length}\t{width}\t{length * width}")
print(f"{9}\t{7}\t{9 * 7}")
print(f"{1}\t{2}\t{1 * 2}\n{3}\t{3}\t{3 * 4}\n{10}\t{2.5}\t{10 * 2.5}")
```

Length	Width	Area
15	3	45
9	7	63
1	2	2
3	3	12
10	2.5	25.0

Formatting number display:

```
pi = 3.14159265359
print(f"{pi:.2f}")
print(f"{pi:.4f}")
```

3.14
3.1416

```
a = 17845005673
print(f"{a:.2e}")
```

1.78e+10

Data structures

The “building block” data types:

Int
Integers

```
animal_legs = 4  
  
type(animal_legs)  
  
int
```

float
floating-point numbers

```
animal_weight = 15.23  
  
type(animal_weight)  
  
float
```

Str
Strings

```
animal_type = "Dog"  
  
type(animal_type)  
  
str
```

Bool
Booleans

```
animal_is_dog = True  
  
type(animal_is_dog)  
  
bool
```

Data structures: containers that store data in an organized manner

Example:

The list data structure:

```
several_integers = [1, 6, 8, 2, 1, 7]  
several_float_numbers = [7.5, 3.1415, 0.4, 10.2]  
several_strings = ["Cat", "Dog", "Apple", "Tree"]  
several_Booleans = [True, False, False, True, True]  
  
print(several_integers)  
print(several_float_numbers)  
print(several_strings)  
print(several_Booleans)  
  
[1, 6, 8, 2, 1, 7]  
[7.5, 3.1415, 0.4, 10.2]  
['Cat', 'Dog', 'Apple', 'Tree']  
[True, False, False, True, True]
```

```
print(type(several_integers))  
print(type(several_float_numbers))  
print(type(several_strings))  
print(type(several_Booleans))  
  
<class 'list'>  
<class 'list'>  
<class 'list'>  
<class 'list'>
```

Data structures

list [a,b,c]

```
animals = ["Cat", "Dog", "Elephant", "Fish"]  
type(animals)
```

list

tuple (a,b,c)

```
cube_1_size = (5, 8, 12)  
type(cube_1_size)
```

tuple

set {a,b,c}

```
cake_ingredients = {"Sugar", "Milk", "eggs", "Flour"}  
type(cake_ingredients)
```

set

dict {key:val, key:val}
(dictionary)

```
animal_color={"Cat":"Black", "Dog":"Brown", "Fish":"Orange"}  
type(animal_color)
```

dict

The `list` data structure

The “`list`” data structure:

an ordered collection of items that can be modified.

`list` `[a,b,c]`

```
animals = ["Cat", "Dog", "Elephant", "Fish"]  
type(animals)
```

```
list
```

Some important methods for the `list` data-structure

Creating a list

```
items = ["A", "B", "C"]
print("Initial list, items: ", items)

Initial list, items:  ['A', 'B', 'C']
```

Appending an item

```
items.append("D")
print(f"After items.append('D') -> items: {items}")

After items.append('D') -> items: ['A', 'B', 'C', 'D']
```

Inserting an item
at a position

```
items.insert(2, "K")
print(f"After items.insert(2, 'K') -> items: {items}")

After items.insert(2, 'K') -> items: ['A', 'B', 'K', 'C', 'D']
```

Pop'ing an item

```
last = items.pop()
print(f"After last = items.pop() -> last = {last} | items: {items}")

After last = items.pop() -> last = D | items: ['A', 'B', 'K', 'C']
```

Removing an item

```
items.remove("K")
print(f"After items.remove('K') -> items: {items}")

After items.remove('K') -> items: ['A', 'B', 'C']
```

Slicing

Slicing enables to subset a sequence of elements

Syntax:

```
sequence(start : end : step)
```

For `start` and `end`:

- If value ≥ 0 then the value indicates the index position
- If value < 0 then the value indicates the index position from the end

Default values:

- `start = 0`
- `end = the last position`
- `step = 1`

When an end position is given, the end position itself is excluded

list: slicing

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
numbers[0:4:1] ➡ [0, 1, 2, 3]
```

```
numbers[:4] ➡ [0, 1, 2, 3]
```

```
numbers[1:4] ➡ [1, 2, 3]
```

```
numbers[-2:] ➡ [9, 10]
```

```
numbers[::2] ➡ [0, 2, 4, 6, 8, 10]
```

```
numbers[::-3] ➡ [10, 7, 4, 1]
```


list: sorting

```
nums = [7, -2, 15, 5, 7, -9, 1, -11, 3, 4]
```

Default sorting (ascending order) `sorted(nums)`

```
[-11, -9, -2, 1, 3, 4, 5, 7, 7, 15]
```

Sorting in reverse order `sorted(nums, reverse = True)`

```
[15, 7, 7, 5, 4, 3, 1, -2, -9, -11]
```

Sorting using a function-derived key (`abs`) `sorted(nums, key = abs)`

```
[1, -2, 3, 4, 5, 7, 7, -9, -11, 15]
```

Data Structure: tuple

The “tuple” data structure: an ordered, immutable collection of items.

tuple (a, b, c)

```
cube_1_size = (5, 8, 12)
type(cube_1_size)
```

```
tuple
```

tuple: Creating, accessing and slicing

Creating a tuple

```
results = (27, 98, 312, 187)
features = ("Green", 54, [5, 3, 90, 1], True)
```

Accessing

```
results[0]
```

```
27
```

Slicing

```
features[0:4:2]
```

```
('Green', [5, 3, 90, 1])
```

Immutability

```
features[0] = "Orange"
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[62], line 1
----> 1 features[0] = "Orange"

TypeError: 'tuple' object does not support item assignment
```

Data Structures: set

The “set” data structure: an unordered collection of unique elements.

set {a, b, c}

```
cake_ingredients = {"Sugar", "Milk", "eggs", "Flour"}  
type(cake_ingredients)
```

```
set
```

Sets are typically used to:

- Ensure uniqueness of elements
- Perform set algebra operations (union, intersection, difference)

Creating a set and adding elements

Creating a set

```
set_1 = {'a', 'b', 'c', 'd'}  
set_1
```

```
{'a', 'b', 'c', 'd'}
```

Adding an element

```
set_1.add("k")
```

```
set_1
```

```
{'a', 'b', 'c', 'd', 'k'}
```

Adding multiple elements

```
set_1.update(["G", "H", "T"])
```

```
set_1
```

```
{'G', 'H', 'T', 'a', 'b', 'c', 'd', 'k'}
```

set: Removing elements

remove

```
set_1 = {'a', 'b', 'c', 'd'}
```

```
set_1.remove("a")
```

```
set_1.remove("k")
```

```
-----  
KeyError  
Cell In[116], line 1  
----> 1 set_1.remove("k")  
  
KeyError: 'k'
```

Remove an element from a set;
The given element must be a member

discard

```
set_1 = {'a', 'b', 'c', 'd'}
```

```
set_1.discard("a")
```

```
set_1.discard("k")
```

```
set_1
```

```
{'b', 'c', 'd'}
```

Discard an element from a set if it is a member

A set does not retain element copies or elements order

```
{"a", "a", "a"}
```

```
{'a'}
```

```
{1,2,3} == {2,2,3,3,3,1}
```

```
True
```

```
{"a", "b", "c"} == {"c", "b", "a"}
```

```
True
```

```
a = {50, 2, 400, 5}  
print(a)
```

```
{400, 50, 2, 5}
```

```
a.add(10)  
print(a)
```

```
{2, 5, 10, 400, 50}
```

set: set operations

```
set_1 = {"a", "b", "c", "d", "e", "f"}  
set_2 = {"e", "f", "g", "h", "i", "j"}
```

Sets intersection

```
set_1 & set_2
```

```
{'e', 'f'}
```

Sets union

```
set_1 | set_2
```

```
{'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j'}
```

Sets unsymmetric difference

```
set_1 - set_2
```

```
{'a', 'b', 'c', 'd'}
```

Sets symmetric difference

```
set_1 ^ set_2
```

```
{'a', 'b', 'c', 'd', 'g', 'h', 'i', 'j'}
```


set: Checking if an element or another set is contained

Checking if an element is contained

```
set_1 = {"a", "b", "c", "d"}
```

```
"c" in set_1
```

True

```
"f" in set_1
```

False

Checking if a set is subset of another set

```
set_2 = {"b", "c"}
```

```
set_2.issubset(set_1)
```

True

```
set_3 = {"b", "c", "e"}
```

```
set_3.issubset(set_1)
```

False

Checking if two sets are disjoint

```
set_4 = {"x", "y", "z"}
```

```
set_4.isdisjoint(set_1)
```

True

```
set_3.isdisjoint(set_1)
```

False

Data Structures: Dictionary

The “dictionary” (dict) data structure:

An unordered collection of unique key-value pairs, where values are mutable and accessed via keys.

key value
Syntax: `{'a': 1, 'b': 2}`

dict {key:val, key:val}
(dictionary)

```
animal_color={"Cat":"Black", "Dog":"Brown", "Fish":"Orange"}  
type(animal_color)
```

dict

dict: Creating a dictionary

Examples of ways to create a dictionary:

Explicitly:

```
d1 = {'a': 1, 'b': 2}
print(d1)

{'a': 1, 'b': 2}
```

Using `dict()`:

```
d2 = dict(c=3, d=4)
print(d2)

{'c': 3, 'd': 4}
```

Using `dict.fromkeys()`:

```
d3 = dict.fromkeys(['e', 'f'], 0)
print(d3)

{'e': 0, 'f': 0}
```

dict: Adding and updating elements

Adding an element

```
d1 = {'a': 1, 'b': 2}
d1['c'] = 3
print(d1)
```

```
{'a': 1, 'b': 2, 'c': 3}
```

Updating an element

```
d1['a'] = 7
d1
```

```
{'a': 7, 'b': 2, 'c': 3}
```

Updating multiple elements

```
d1 = {'a': 1, 'b': 2}
d1.update({'c': 3, 'b': 17})
print(d1)
```

```
{'a': 1, 'b': 17, 'c': 3}
```

dict: Accessing an element based on key

Simple access

```
d1 = {'a': 5, 'b': 17, 'c': 7}
```

```
d1['b']
```

```
17
```

```
d1['g']
```

```
-----  
KeyError  
Cell In[93], line 1  
----> 1 d1['g']  
  
KeyError: 'g'
```

If the key is missing
→ Error

Using .get

```
d1 = {'a': 5, 'b': 17, 'c': 7}
```

```
d1.get('b', 'not found')
```

```
17
```

```
d1.get('g', 'not found')
```

```
'not found'
```

```
d1.get('g')
```

If the key is
missing, returns
the given default

dict: Deleting an element

Deleting an element using `del`

```
d1 = {'a': 1, 'b': 17, 'c': 3}
```

```
del d1['b']
```

```
d1
```

```
{'a': 1, 'c': 3}
```

```
del d1['g']
```

```
-----  
KeyError  
Cell In[51], line 1  
----> 1 del d1['g']  
KeyError: 'g'
```

Deletes the element
based on the key

If the key is missing
→ Error

Deleting an element using `.pop()`

```
d1 = {'a': 1, 'b': 17, 'c': 3}
```

```
d1.pop('b', None)
```

```
17
```

```
d1
```

```
{'a': 1, 'c': 3}
```

```
d1.pop('g', None)
```

```
d1.pop('g', 'not found')
```

```
'not found'
```

Deletes the element
based on the key
and returns its value

If the key is
missing, returns
the given default

Data Structures: Summary

	list	tuple	set	dict
Syntax	[1,2,3]	(1,2,3)	{1,2,3}	{'a': 1, 'b': 2, 'c': 3}
Ordered	Yes	Yes	No	Insertion ordered
Mutable	Yes	No	Yes	Yes (keys immutable)
Allows dupliptes	Yes	Yes	No	Keys no, Values Yes
Indexing / Slicing	Yes	Yes	No	No numeric indexing / slicing
Typical use	Ordered mutable sequence	Fixed read-only sequence	Unique items & Set operations	Key → value mapping

Mutability of objects in Python

Variables are names (labels) that reference objects in memory

Mutability: whether an object can be changed after it is created

Immutable objects cannot be modified.

➤ Therefore, any “change” in it actually creates a new object.

Immutable objects:

- int
- float
- str
- tuple

Mutable objects can be modified in place.

➤ Therefore, changes in such an object affect all the references to it. (!!!)

Exmplr for mutable objects:

- list
- dict
- set

Mutability of objects in Python

Immutable object (e.g. `int`)

```
a = 10
b = a      # b refers to the same value as a

b = b + 1   # create a NEW integer and reassign b

print(f"a: {a}")
print(f"b: {b}")

a: 10
b: 11
```

Mutable object (e.g. `list`)

```
a = [1, 2]
b = a      # b refers to the SAME list as a

b.append(3) # modify the shared list

print(f"a: {a}")
print(f"b: {b}")

a: [1, 2, 3]
b: [1, 2, 3] ← Also changed ! (not good)
```

```
a = [1, 2]
b = a.copy() # make a NEW, separate list with
              # the values of a

b.append(3)   # modify list b in place

print(f"a: {a}")
print(f"b: {b}")

a: [1, 2]
b: [1, 2, 3] 
```

The range () function

The range () function enables to generate a sequence of equally-spaced integers

Syntax:

```
range(start : end : step)
```

start: Start value (default =0)
end: End value (not included !)
step: Increment (can be negative) default = 1

The range () function returns a range object

```
range(10,20,2)
```

```
range(10, 20, 2)
```

The range object can be directly converted to list / set / tuple
Using the list(), set() or tuple() functions

```
list(range(10, 20, 2))
```

```
[10, 12, 14, 16, 18]
```

Examples:

```
list(range(1, 11))
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
list(range(20, 10, -3))
```

```
[20, 17, 14, 11]
```

```
list(range(5))
```

```
[0, 1, 2, 3, 4]
```

Converting between set, list, tuple

```
my_list = ["a", "b", "c", "d"]
my_tuple = ("a", "b", "c", "d")
my_set = {"a", "b", "c", "d"}
```

Converting to list
list()

```
list(my_tuple)
```

```
['a', 'b', 'c', 'd']
```

```
list(my_set)
```

```
['d', 'a', 'c', 'b']
```

Converting to tuple
tuple()

```
tuple(my_list)
```

```
('a', 'b', 'c', 'd')
```

```
tuple(my_set)
```

```
('d', 'a', 'c', 'b')
```

Converting to set*
set()

```
set(my_list)
```

```
{'a', 'b', 'c', 'd'}
```

```
set(my_tuple)
```

```
{'a', 'b', 'c', 'd'}
```

*Note that in conversion to tuple, order and repetitions are not retained !

Control Flow: if / elif / else

Execute code based on conditions

```
if (condition):  
    statement to do  
else:  
    statment to do
```

```
if (condition):  
    statement to do  
elif:  
    statment to do  
else:  
    statment to do
```

In Python indentation defines code blocks and hierarchy

Condition control flow: if / elif / else

```
chairs =2
people = 3

# Example 1:
if people <= chairs:
    print(f"There are enough chairs")
else:
    print(f"There are not enough chairs")

# Example 2:
if people < chairs:
    print(f"There are more chairs than people")
elif people > chairs:
    print(f"There are more people than chairs")
else:
    print(f"Number of people is the same as the numebr of chairs")
```

There are not enough chairs
There are more people than chairs

Conidtion control flow: match, case

`match, case`: checks a value against patterns and executes the first matching block

```
place = 3

match place:
    case 1:
        print("First")
    case 2:
        print("Second")
    case 3:
        print("Third")
    case _:
        print("After the third place")
```

Third

```
day = "Friday"

match day:
    case "Saturday":
        print("weekend starts")
    case "Sunday":
        print("weekend ends")
    case _:
        print(f"{day} is a working day")
```

Friday is a working day

Course: Hands-On Data Analytics with Python

Eli Zamir

Lesson 2: Core Python programming (part 1)

Hands-on session

Exercise 1 (practicing a variety of aspects)

(1) Using `range(start:end:step)` create 3 lists:

"a100" : of all integers from 1 to 100

"a3": of all integers from 1 to 100 which can be divided by 3 (with no remainder)

"a4": of all integers from 1 to 100 which can be divided by 4 (with no remainder)

(2) Convert these 3 lists to corresponding 3 sets: "s100", "s3", "s4"

(3a) Generate a set "s_3n4" which contains all the elements which are in "s3" AND "s4" (&)

(3b) Generate a set "s_3u4" which contains all the elements which are in "s3" OR "s4" (or in both) (|)

(3c) Generate a set "s_3sd4" which contains all the elements which are either in "s3" or in "s4" (^)

(3d) Generate a set "s_3m4" which contains all the elements which are in "s3" but not "s4" (-)

(4) Check (using `len`) the number of elements in each of the 7 generated sets

(5a) Make a **list** "all_sets" containing all the 7 generated sets

(5b) Generate a version of this list, "all_sets_sorted", in which the sets are sorted by their number of elements

(5c) Check the number of elements in the first set in "all_sets" and in "all_sets_sorted"

(5d) Check the number of elements in last set in "all_sets" and in "all_sets_sorted"

Exercise 2 (adding elements and mutability aspects)

Exercise 2a

- (1) Create a set "letters_set" containing the elements "a", "b", "c".
- (2) Assign and execute: `letters_set_new = letters_set`
- (3) Add to the set `letters_set_new` the element "d"
- (4) Check which elements are now `letters_set`

Exercise 2b

Repeat all the steps of exercise 2a, but in step 2 assign instead by:

```
letters_set_new = letters_set.copy()
```

Exercise 2c

Repeat 2a and 2b, but with lists, instead of sets (and name them `letters_list` and `letters_list_new`)

Exercise 3 (f-string)

- (1) Import numpy as np, and create a variable pi:

```
import numpy as np
pi = np.pi
pi
```

3.141592653589793

- (2) Use **f-string**, and a **variable** for the number of decimal digits to print these statements:

The print statement should be always the same

```
n = 0
print(  )
```

Pi with 0 decimal digits is 3

```
n = 2
print(  )
```

Pi with 2 decimal digits is 3.14

```
n = 5
print(  )
```

Pi with 5 decimal digits is 3.14159

Exercise 4 (condition control flow)

- (1) Write an `if/elif/else` statement that checks if variable `x` is bigger / equal or smaller than 100
- (2) Write a `match/ case` statement that checks the type of variable `y` (using `type()`), and print "1", "2", "3" or "4" if `y` is a list, tuple, dictionary or a set, respectively